Legal Rights Navigator

System Architecture & Build Roadmap

An AI agent that takes a plain-language description of a person's situation, classifies it into the right area of Indian law, retrieves grounded legal context, and produces a structured response — explanation, evidence checklist, recommended authority, or a drafted complaint/notice.

1 · Request Pipeline

Every query moves through a validation → classification → retrieval → planning → generation chain. Each stage exists to keep the expensive LLM calls reserved for genuinely hard decisions.

1. **Auth & Session**
   Google OAuth login → session cookie issued → verified on each request.
2. **Validity Gate (3-tier)**
   Keyword blocklist (regex) → embedding similarity vs. a legal-anchor set → LLM binary check only for ambiguous cases. Invalid queries are simply not persisted to conversation state — not "removed" after the fact.
3. **Domain Classifier**
   Cheap/fine-tuned LLM sorts the query into one of 5 legal domains and returns a confidence score.
4. **Clarification Loop**
   Pulls known facts from the conversation; if critical facts are missing, the system asks a targeted follow-up instead of guessing.
5. **Domain-Filtered Retrieval**
   Vector search restricted to the classified domain's collection first, then reranked with a cross-encoder — prevents a labour-law chunk from leaking into a cybercrime query on raw cosine similarity.
6. **Legal Planner**
   Reasoning LLM decides the response type — explain the law, draft a complaint, generate a legal notice, draft an email, or recommend filing with a specific authority — and emits this as a JSON action plan.
7. **Generation**
   Action plan branches to either an Advice Generator (explanation + checklist + authority) or a template-based Complaint/Notice Generator.
8. **Structured Response**
   Final answer is returned as structured JSON so the frontend can render it predictably.

2 · Stack Rationale

Layer	Choice	Why
Frontend → Backend	React → FastAPI (Python)	AI orchestration (LangGraph) is Python-native. Avoiding a split JS/Python backend removes a whole class of integration overhead.
Framework	FastAPI over Flask	Async by default — concurrent legal queries from multiple users don't block each other. Flask is synchronous and queues requests.
Vector Store	PostgreSQL + pgvector	One database for relational data and vectors, with native metadata filtering. Specialized stores (Qdrant, Milvus, Weaviate) add infra overhead this project doesn't need yet.

3 · The 5 Legal Domains

Each domain owns its own retrieval collection, prompt template, fact checklist, and document generator. This isolation is what makes the domain-filtered retrieval step in the pipeline work.

Labour & Employment	Unpaid wages, wrongful termination, harassment at work.
Consumer Protection	Defective goods, service disputes, refund/Consumer Commission cases.
Tenant & Property	Residential leases, deposits, eviction, maintenance disputes.
Cyber Crime & Online Fraud	Financial fraud, identity theft, harassment online.
Family & Women's Rights	Domestic matters, protection laws, family disputes.

4 · Data Model (Conceptual)

Each Google-authenticated user owns multiple conversations; each conversation belongs to one domain and carries its own rolling summary plus a live, structured set of extracted case facts (not the same as "missing information," which is transient and never stored). Each message logs which model handled it, what legal sources were cited, and which retrieval chunks were used — needed later for both debugging and audit trails.

- **Users** — identity from Google OAuth (permanent Google subject ID, email, name).
- **Conversations** — one per case thread; tagged with domain, auto-generated title, rolling summary, and live extracted facts.
- **Messages** — full turn history with metadata: domain, plan type, sources cited, model used, retrieval chunk IDs.

5 · Knowledge Ingestion & Model Assignment

Legal source documents are chunked, embedded, and stored per-domain ahead of time, independent of the live query pipeline. Different stages deliberately use different models, matched to how much reasoning each step actually needs:

- **Ingestion:** source docs → chunk → embed → store, run as an offline pipeline per domain.
- **Classification:** small/cheap classifier model.
- **Clarification & reasoning:** main chat/reasoning model.
- **Structured output:** same reasoning model, constrained to JSON.
- **Retrieval:** dedicated embedding model + cross-encoder reranker.

This document is a planning artifact, not a contract — refine domain boundaries and model choices as real query data comes in.

Build Checkpoints

Six phases, sequenced by dependency — each one should be demoable before moving on.

1 Foundations: Auth + Skeleton

Goal: A logged-in user can hit a working, empty pipeline.

- Google OAuth login, cookie-based session, verification middleware
- React frontend scaffold with chat UI shell
- FastAPI backend skeleton with one async endpoint
- Postgres schema live: users, conversations, messages

2 Query Validity & Domain Classification

Goal: The system can tell a legal question from noise and route it correctly.

- Tier-1 keyword blocklist for obviously invalid queries
- Tier-2 embedding similarity check against a legal-anchor set
- Tier-3 LLM fallback for ambiguous edge cases
- Domain classifier returning {domain, confidence} for the 5 categories
- Decision rule for low-confidence classifications

3 Knowledge Base & Retrieval

Goal: Relevant law can actually be pulled back for a given query.

- Source documents collected per domain
- Ingestion pipeline: chunk → embed → store in pgvector, tagged by domain
- Domain-filtered semantic search
- Cross-encoder reranking on top of filtered results
- Manual spot-check: does retrieval surface the right chunks for known test queries?

4 Facts, Clarification & Planning

Goal: The agent knows what it doesn't know, and decides what to do next.

- Fact-extraction step updating a conversation's live fact set each turn
- Missing-information detection that triggers a clarifying question instead of guessing
- Legal Planner producing a structured action plan: explain / draft complaint / draft notice / recommend authority
- LangGraph state wired: query, domain, confidence, facts, retrieved docs, plan, response

5 Response Generation

Goal: The plan turns into a real, structured answer.

- Advice Generator: explanation + evidence checklist + authority recommendation
- Template-based Complaint/Notice Generator per domain (forms drafted per domain)
- All outputs returned as structured JSON consumable by the React frontend
- Message metadata logging: model used, sources cited, chunk IDs

6 Conversation Memory & Polish

Goal: Multi-turn conversations hold up and the product feels finished.

- Rolling conversation summarization once message_count crosses a threshold
- Conversation list/history UI per user
- End-to-end test pass across all 5 domains with realistic queries
- Deployment + basic monitoring for failure cases (low-confidence classifications, empty retrievals)